段冰 Reese

+86 18510840710 | reese_duan@163.com | github.com/imReese | 技术博客

求职方向 AI Infra / 大模型推理系统

聚焦大模型推理系统与性能优化，涵盖运行时调度、状态存储与复用，以及并行执行和传输协同；具备 CPU 微架构性能分析和分布式存储控制面开发经验。

工作经历

百度 · AI 计算 / 训推引擎组 2025.06 - 至今

软件工程师（大模型推理引擎）

负责 SGLang 生产级部署与工程化，支撑大规模集群推理服务；完善引擎可观测性接口，定位并解决核心稳定性问题。

- **缓存索引与存储子系统**：完成 Attention Store（AS）缓存与统一 radix 树的合并迁移，建成 AS 存储子系统；修复多并行、多模态场景的稳定性问题。
- **模型适配与并行部署**：完成 Kimi 系列引擎适配，处理 PP 部署适配与稳定性问题；支撑 GLM、DeepSeek 系列落地与精度验证。团队 AS 方案支持 KDA 状态及 PP 下的 L3 存储接入。
- **缓存读写与复用收益**：优化 Attention Store 读写与前缀复用，提升缓存命中率和 Prefill 吞吐。
- **传输与 GPU 批次执行**：优化主机侧执行、KV cache 操作与 GPU 批次执行之间的异步流程，实现 H2D、D2H 传输与计算重叠；结合 Mooncake TransferEngine 分析与优化 KV 传输路径。
- **推理执行与调度优化**：承担 Prefill/EPD 与约束解码相关优化；对比定长压测与真实流量，分析请求长度和 KDA 后端选择对 Prefill 性能及缓存复用收益的影响。
- **分层缓存架构验证**：探索 LMCache 与 KV manager 的分层架构，搭建 LMCache 服务端并完成方案可行性验证。

华为云 · 竖亥实验室 2023.07 - 2024.07

计算机体系结构研究工程师

- **负载分析**：基于 perf 和 Intel Pin 构建低开销指令流分析工具，提取 MySQL、Redis 等云服务负载的指令流特征，为微架构方案验证提供依据。
- **微架构验证**：结合指令流特征与 ChampSim 仿真验证 CPU 方案，TLB 命中率提升 30%，IPC 提升 6%；相关方案已申请发明专利。
- **软硬协同**：与海思、openEuler 团队协作验证鲲鹏 CPU 性能优化方案，单核吞吐量提升 8%。

华为数据存储产品线 · 基础设施开发部 2022.05 - 2023.07

软件开发工程师

- **控制面开发**：负责分布式存储 Cluster Manager，覆盖服务选主、客户端选主与集群生命周期管理。
- **异步化与规模扩展**：通过异步消息队列和模块解耦，将单集群支持节点数从 128 扩展至 256，资源初始化效率提升 40%。
- **可靠性建设**：主导 200+ 接口自动化测试与安全加固，分支覆盖率 90%+；定位并解决 20+ 生产环境高优先级问题。

段冰 Reese AI Infra / 大模型推理系统

独立系统项目

独立开展推理运行时、状态复用系统和推理控制面的设计与实现。

SGLang Rust Runtime 2026.05 - 至今
github.com/imReese/sglang-rs Rust / gRPC / CUDA

- **批次调度**：按未缓存 token 数控制 Prefill 批次规模，支持页对齐的前缀复用；P/D 分离模式下，Decode 等待 KV 就绪后执行，异步 Prefill 的缓存页在传输完成后才对外可用。
- **统一物理 KV 页**：CUDA 执行与 Mooncake 注册共用同一 page-major KV pool，通过 RAII 管理注册和设备内存，确保先注销再释放；失败时回收尚未对外提供的缓存页，请求取消时同步取消相关传输。
- **执行后端**：实现 Qwen2/3 共享 dense decoder、F32 CPU 参考与 BF16 CUDA 执行路径，接入 cuBLAS GEMM 与 GQA paged attention。

NexusKV 2025.04 - 至今
github.com/imReese/NexusKV Go / Rust / Python

- **缓存复用判断**：设计跨引擎的状态复用方案：命中后检查租户、模型版本和数据布局是否匹配，再评估复用范围、传输成本及引擎的使用条件，并记录判断依据。
- **引擎与存储解耦**：设计统一的状态描述和复用接口，将复用判断与存储、传输分开；由推理引擎决定是否使用外部缓存并负责内存分配，便于接入不同引擎和存储后端。
- **索引与引擎适配**：使用 Rust 实现前缀匹配和写时复制索引，构建 Host DRAM 存储；通过引擎适配接口协调状态传输、使用与释放。

Locus
github.com/imReese/Locus Rust / Axum

- **跨实例调度**：设计推理控制面，结合模型语义和引擎能力筛选可用实例，再综合租户策略、实例负载与缓存位置，选择请求的执行实例。
- **规划与执行分离**：控制面负责跨实例调度，引擎负责实例内批次调度；将实例选择与引擎调用、状态传输分开，便于独立调整调度策略并管理执行过程。
- **执行流程**：构建 OpenAI/Anthropic 兼容接口和租户准入机制，串联目标资源预留、状态导入协调与任务提交；执行失败时按预定范围回退并清理资源。

架构设计建议

向 SGLang 提出缓存数据面与 Scheduler 分离的 RFC（#37372），讨论元数据所有权、GPU 内存 IPC 与生命周期；另建议将共享缓存传输类型迁到中立模块并兼容导出（#38889）。

技术技能

编程语言：C/C++、Python、Go、Rust、Shell
性能与系统：Linux、perf、Intel Pin、ChampSim、gdb、Paxos
工程工具：Docker、Kubernetes、Prometheus、Grafana、Git、CMake

教育背景

南京大学 2017.09 - 2021.06
环境工程 · 学士 GPA 4.2/5.0